

LAB Assignment No. 11

Case Study: Smart Home Automation System using ESP32, Firebase, and Web Dashboard (HTML, CSS, JavaScript)

1. Introduction

A Smart Home Automation System is an Internet of Things (IoT)-based solution that enables users to remotely monitor and control household appliances through internet-connected devices. The system provides:

- Real-time monitoring
- Remote appliance control
- Energy efficiency
- Improved security
- Intelligent automation

This project is developed using:

- ESP32 microcontroller
- Wokwi simulation platform
- Firebase Realtime Database
- Web Dashboard using HTML, CSS, and JavaScript
- ThingSpeak cloud analytics

The smart home system allows users to monitor sensors and control home appliances from a web browser in real time.

2. Objectives of the Project

The major objectives are:

1. To develop an IoT-based smart home automation system.
2. To simulate the project using Wokwi.
3. To design a responsive web dashboard using HTML, CSS, and JavaScript.
4. To integrate Firebase Realtime Database for cloud communication.

5. To monitor sensor data remotely.
6. To control home appliances through a web interface.
7. To visualize sensor data using ThingSpeak.

Problem Statement

Traditional home systems rely on manual control and lack remote monitoring capabilities. This creates several problems:

- Energy wastage
- Lack of automation
- Limited security
- No remote accessibility
- Difficulty in monitoring appliances

The proposed IoT smart home system addresses these problems by integrating ESP32 with cloud-based technologies and a real-time web dashboard.

4. Proposed Solution

The proposed solution consists of:

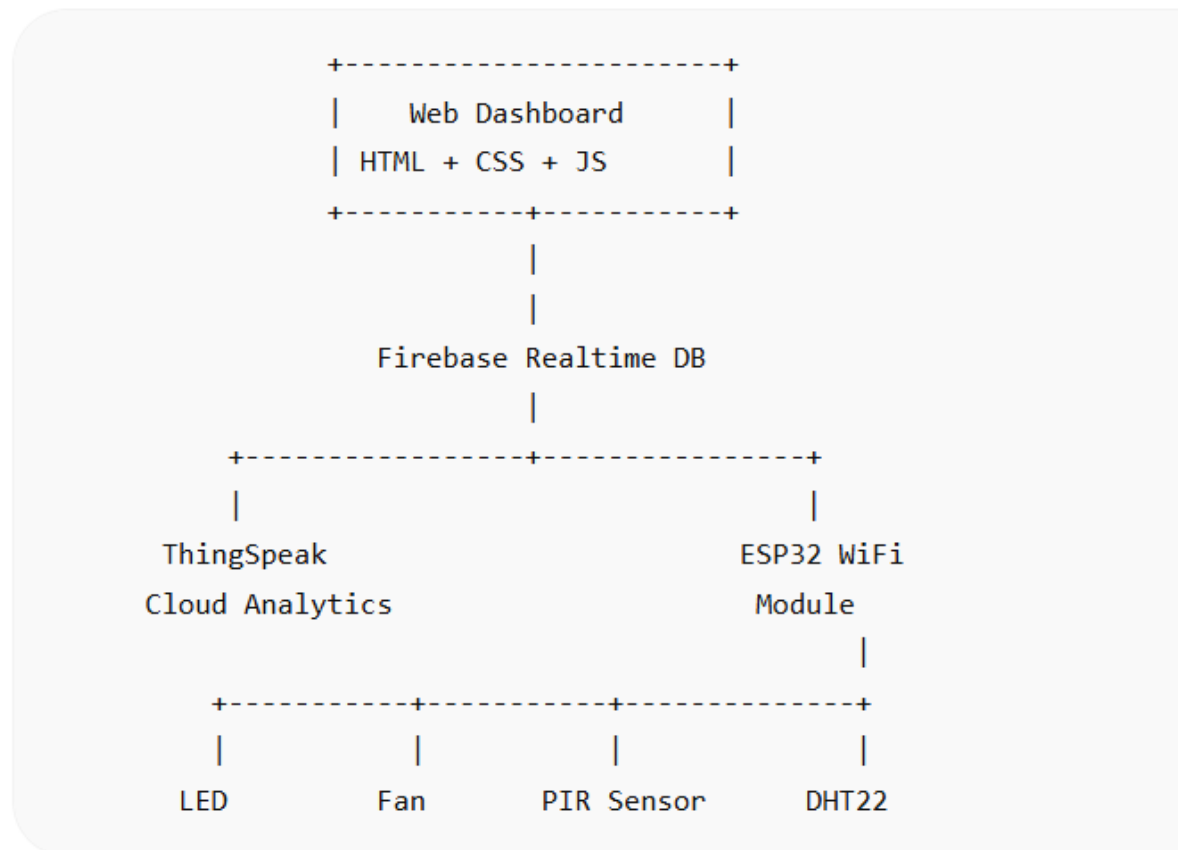
Component	Function
ESP32	Main IoT controller
Firebase	Real-time cloud database
HTML/CSS/JS Dashboard	Web interface
ThingSpeak	Cloud analytics
Sensors	Environmental monitoring
Relays/LEDs	Appliance control
Wokwi	Online simulation

The system enables:

- Real-time monitoring
- Remote control of devices
- Cloud-based data storage
- Responsive web dashboard visualization

5. System Architecture

Overall System Architecture



6. Hardware Components

Component	Description
ESP32	WiFi-enabled controller
DHT22	Temperature and humidity sensor
PIR Sensor	Motion detection
MQ2 Sensor	Gas detection
Relay Module	Appliance switching
LED	Light simulation
Buzzer	Alarm indication
Breadboard	Connections
Jumper Wires	Circuit wiring

7. Software Requirements

Software	Purpose
Arduino IDE	ESP32 programming
Wokwi	IoT simulation
Firebase	Cloud database
HTML	Web structure
CSS	Dashboard styling
JavaScript	Dashboard logic
ThingSpeak	Data analytics

8. Wokwi Simulation

The entire smart home system is simulated using: [Wokwi](#)

Simulated Devices

- ESP32
- LEDs
- DHT22 sensor
- PIR sensor
- Relay module

Advantages of Wokwi

- No physical hardware required
- Easy debugging
- Real-time serial monitor
- Fast testing environment

9. ESP32 Functional Description

The ESP32 performs the following operations:

Sensor Monitoring

The ESP32 continuously reads:

- Temperature
- Humidity
- Motion detection
- Gas leakage
- Appliance Control

The ESP32 controls:

- Lights
- Fan
- Alarm buzzer

Cloud Communication

The ESP32 sends sensor data to:

- Firebase Realtime Database
- ThingSpeak Cloud

10. Firebase Integration

[Firebase](#) is used as the central cloud database for real-time communication.

Firestore Features

- Realtime synchronization
- Cloud storage
- Device control
- Web integration
- Authentication support

Example Firestore Data Structure

JSON

```
{
  "SmartHome": {
    "Temperature": 29,
    "Humidity": 65,
    "Motion": 1,
    "Gas": 0,
    "Light": 1,
    "Fan": 0
  }
}
```

11. Web Dashboard using HTML, CSS, and JavaScript

Dashboard Overview

The web dashboard is developed using:

- HTML → Structure
- CSS → Styling
- JavaScript → Firebase communication

The dashboard provides:

- Appliance control buttons
- Real-time sensor monitoring
- Device status display
- Responsive interface

12. Dashboard Features

Feature	Description
Light Control	Turn lights ON/OFF
Fan Control	Control cooling system
Temperature Display	Real-time monitoring
Motion Status	Intruder detection
Gas Alert	Leakage warning
Live Updates	Firebase synchronization

13. HTML Dashboard Structure

Main Sections

```
</> HTML

<header>
  Smart Home Dashboard
</header>

<div class="card">
  Temperature Display
</div>

<div class="card">
  Light Control Button
</div>

<div class="card">
  Fan Control Button
</div>
```

14. CSS Styling Features

User Interface Design

CSS provides:

- Responsive layout
- Card-based design

- Modern UI
- Mobile compatibility
- Animated buttons

Example Styling

`</>` CSS

```
.card {  
  background: white;  
  border-radius: 10px;  
  padding: 20px;  
}
```

15. JavaScript Functionality

Firestore Communication

JavaScript performs:

- Reading sensor data
- Updating device status
- Sending ON/OFF commands
- Real-time synchronization

Example JavaScript Logic

```
⌞⌟ JavaScript
```

```
firebase.database().ref("SmartHome/Light")  
.set(1);
```

16. Dashboard Working Principle

Step-by-Step Operation

Step 1: ESP32 Connects to WiFi

The ESP32 establishes internet connectivity.

Step 2: Sensor Data Collection

ESP32 reads sensor values:

- Temperature
- Motion
- Gas status

Step 3: Firebase Upload

Sensor data is uploaded to Firebase Realtime Database.

Step 4: Dashboard Synchronization

JavaScript fetches real-time data from Firebase.

Step 5: User Controls Appliances

The user clicks buttons on the dashboard.

Step 6: Firebase Updates ESP32

ESP32 reads updated commands and controls appliances.

17. ThingSpeak Cloud Analytics

Introduction

[ThingSpeak](#) is used for IoT cloud analytics and graph visualization.

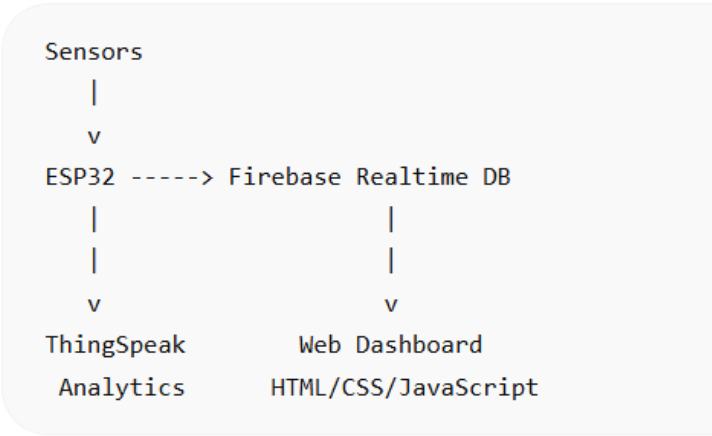
ThingSpeak Features

- Real-time graphs
- Data logging
- Historical analysis

Example Fields

Field	Data
Field 1	Temperature
Field 2	Humidity
Field 3	Motion
Field 4	Gas sensor

18. Data Flow Diagram



19. Automation Scenarios

Smart Lighting

Motion Detected → Light ON
No Motion → Light OFF

Smart Cooling

Temperature > 35°C → Fan ON

Gas Leakage Alert

Gas Detected → Alarm ON

20. Advantages of the Proposed System

Technical Advantages

- Real-time cloud communication
- Remote appliance control
- Responsive dashboard
- Wireless operation
- Easy scalability

21. Limitations

Limitation	Description
Internet Dependency	Requires WiFi
Security Risks	Cloud vulnerabilities
Sensor Errors	Environmental effects
Power Failure	System interruption

22. Future Enhancements

Future improvements may include:

- AI-based automation
- Mobile application
- Voice assistant integration
- Face recognition
- Smart energy prediction
- Machine learning analytics

23. Applications

Residential

- Smart homes
- Apartments
- Villas

Commercial

- Offices
- Hotels

- Smart buildings

Industrial

- Factory automation
- Security systems

Conclusion

The Smart Home Automation System using ESP32, Firebase, HTML, CSS, JavaScript, and ThingSpeak demonstrates a powerful IoT solution for intelligent home management.

The system provides:

- Real-time monitoring
- Remote control
- Web-based accessibility
- Cloud analytics
- Automation capabilities

The project is scalable, cost-effective, and suitable for modern smart home applications.

Review Questions

Question 1

Explain the concept of Smart Home Automation Systems and discuss their importance in modern society.

Question 2

Discuss the role of the following technologies in the proposed system:

Technology	Discussion Required
ESP32	WiFi communication
Firebase	Real-time cloud database
Wokwi	Simulation platform
HTML/CSS/JS	Dashboard development
ThingSpeak	IoT analytics

Task 2: System Architecture

Question 3

Design the overall architecture diagram of the Smart Home Automation System.

Requirements:

- ESP32
- Sensors
- Firebase
- Web dashboard
- ThingSpeak

- WiFi communication

Submission Required:

- Architecture diagram screenshot
- Brief explanation

Question 4

Create the complete smart home automation circuit in [Wokwi](#) using ESP32.

Question 5

Write the ESP32 Arduino code to:

- Connect ESP32 with WiFi
- Read sensor values
- Control appliances
- Send data to Firebase
- Send data to ThingSpeak

Submission Requirements

Students must submit:

- Arduino code
- Wokwi simulation link
- Circuit screenshot
- Serial monitor screenshot

Question 6

Create a Firebase Realtime Database for the smart home system.

Students must perform:

- Firebase project creation
- Realtime database setup
- JSON data creation
- Database rules configuration

Submission Requirements

Students must provide:

- Firebase project screenshots
- Database JSON structure screenshot
- Explanation of database nodes

Task 5: Web Dashboard Development

Question 7

Develop a Smart Home Web Dashboard using:

- HTML
- CSS
- JavaScript

Dashboard Requirements

The dashboard must include:

Feature	Requirement
Temperature Display	Real-time update
Motion Status	Detection status
Light Control	ON/OFF button
Fan Control	ON/OFF button
Gas Alert	Warning message
Responsive Design	Mobile compatible

Question 8

Explain how JavaScript communicates with Firebase Realtime Database.

Students should discuss:

- Firebase SDK
- Real-time listeners
- Data synchronization
- Cloud communication

Submission Requirements

Students must submit:

- HTML code
- CSS code
- JavaScript code
- Dashboard screenshots
- Working demonstration screenshots

Task 6: ThingSpeak Cloud Analytics

Question 9

Configure [ThingSpeak](#) to visualize sensor data.

Students must:

- Create ThingSpeak channel
- Configure fields
- Send ESP32 sensor data
- Generate real-time graphs

Submission Requirements

Students must submit:

- ThingSpeak channel screenshots
- Graph screenshots
- API configuration screenshot

Task 7: Automation Logic

Question 10

- Implement the following automation scenarios:

Scenario 1

```
IF Motion Detected  
THEN Light ON
```

Scenario 2

```
IF Temperature > 35°C  
THEN Fan ON
```

Scenario 3

```
IF Gas Leakage Detected  
THEN Alarm ON
```

Question 11

Suggest future improvements for the Smart Home Automation System.

Possible topics:

- AI integration
- Voice assistant
- Mobile app
- Face recognition
- Energy optimization

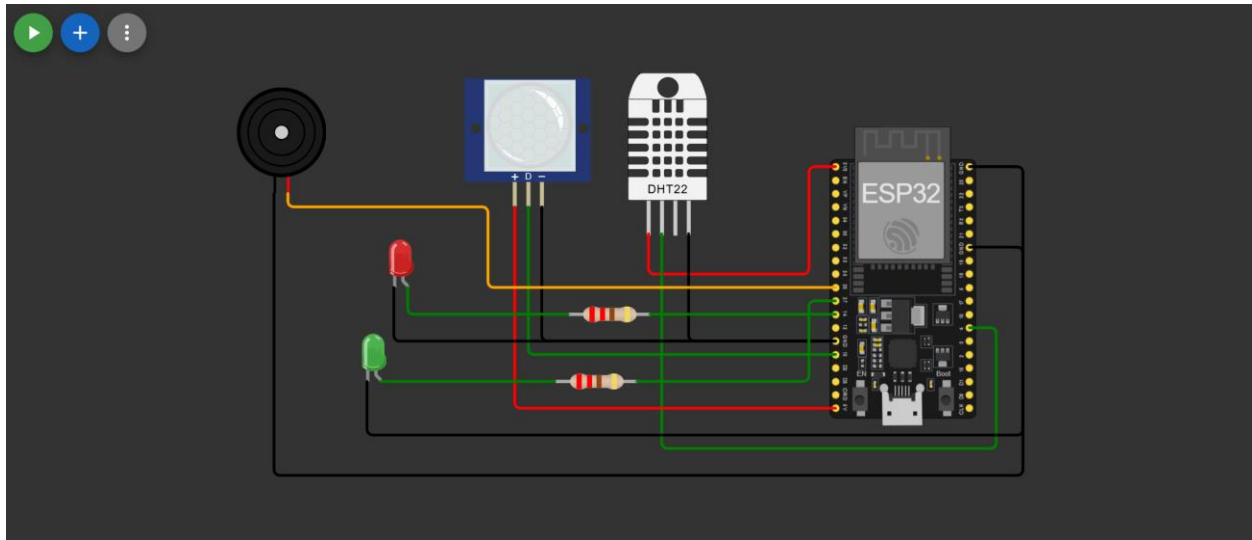
Mandatory Submission Components

Students must submit the following:

Item	Required
Assignment report	Yes
ESP32 Arduino code	Yes
HTML/CSS/JS code	Yes
Wokwi simulation screenshots	Yes
Firebase screenshots	Yes
ThingSpeak screenshots	Yes
Dashboard screenshots	Yes
Flowcharts	Yes
Architecture diagram	Yes
Working explanation	Yes



SOLUTION:



C++ Code

```

1  #include <WiFi.h>
2  #include <DHT.h>
3  #include <ThingSpeak.h>
4
5  // ===== WiFi =====
6  #define WIFI_SSID "Wokwi-GUEST"
7  #define WIFI_PASSWORD ""
8
9  // ===== ThingSpeak =====
10 #define TS_CHANNEL_ID 3376769
11 #define TS_WRITE_API_KEY "FFKFYPWWYC07P2SJ"
12 #define TS_READ_API_KEY "D48ZAHQ3AX1906P1"
13
14 // You only have 3 fields:
15 // Field 1 = Temperature
16 // Field 2 = Humidity
17 // Field 3 = Motion
18 //
19 // LED control is handled only from the web dashboard
20 // ===== Pins =====
21 #define DHTPIN 4
22 #define DHTTYPE DHT22
23 #define PIR_PIN 13
24 #define LIGHT_LED 14
25 #define FAN_LED 27

```

```

26  #define BUZZER 26
27
28  // ===== Objects =====
29  DHT dht(DHTPIN, DHTTYPE);
30  WiFiClient client;
31
32  // ===== Timing =====
33  unsigned long lastUpload = 0;
34  const unsigned long uploadInterval = 15000; // ThingSpeak minimum interval
35
36  void connectWiFi() {
37      if (WiFi.status() == WL_CONNECTED) return;
38
39      Serial.print("Connecting to WiFi");
40      WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
41
42      while (WiFi.status() != WL_CONNECTED) {
43          delay(500);
44          Serial.print(".");
45      }
46
47      Serial.println();
48      Serial.println("WiFi Connected");
49      Serial.print("IP Address: ");
50      Serial.println(WiFi.localIP());

```

```

53  void setup() {
54      Serial.begin(115200);
55
56      pinMode(PIR_PIN, INPUT);
57      pinMode(LIGHT_LED, OUTPUT);
58      pinMode(FAN_LED, OUTPUT);
59      pinMode(BUZZER, OUTPUT);
60
61      digitalWrite(LIGHT_LED, LOW);
62      digitalWrite(FAN_LED, LOW);
63      digitalWrite(BUZZER, LOW);
64
65      dht.begin();
66
67      connectWiFi();
68
69      ThingSpeak.begin(client);
70
71      Serial.println("System Ready");
72  }
73
74  void loop() {
75      connectWiFi();
76

```

```

76
77 // ===== Read Sensors =====
78 float humidity = dht.readHumidity();
79 float temperature = dht.readTemperature();
80 int motion = digitalRead(PIR_PIN);
81
82 if (isnan(humidity) || isnan(temperature)) {
83     Serial.println("Failed to read DHT22");
84     delay(2000);
85     return;
86 }
87
88 // ===== Local Automation =====
89 // Light LED turns ON when motion is detected
90 digitalWrite(LIGHT_LED, motion ? HIGH : LOW);
91
92 // Fan turns ON when temperature > 30°C
93 digitalWrite(FAN_LED, temperature > 30 ? HIGH : LOW);
94
95 // Buzzer turns ON when motion is detected
96 digitalWrite(BUZZER, motion ? HIGH : LOW);
97
98 // ===== Upload to ThingSpeak =====
99 if (millis() - lastUpload >= uploadInterval) {

```

```

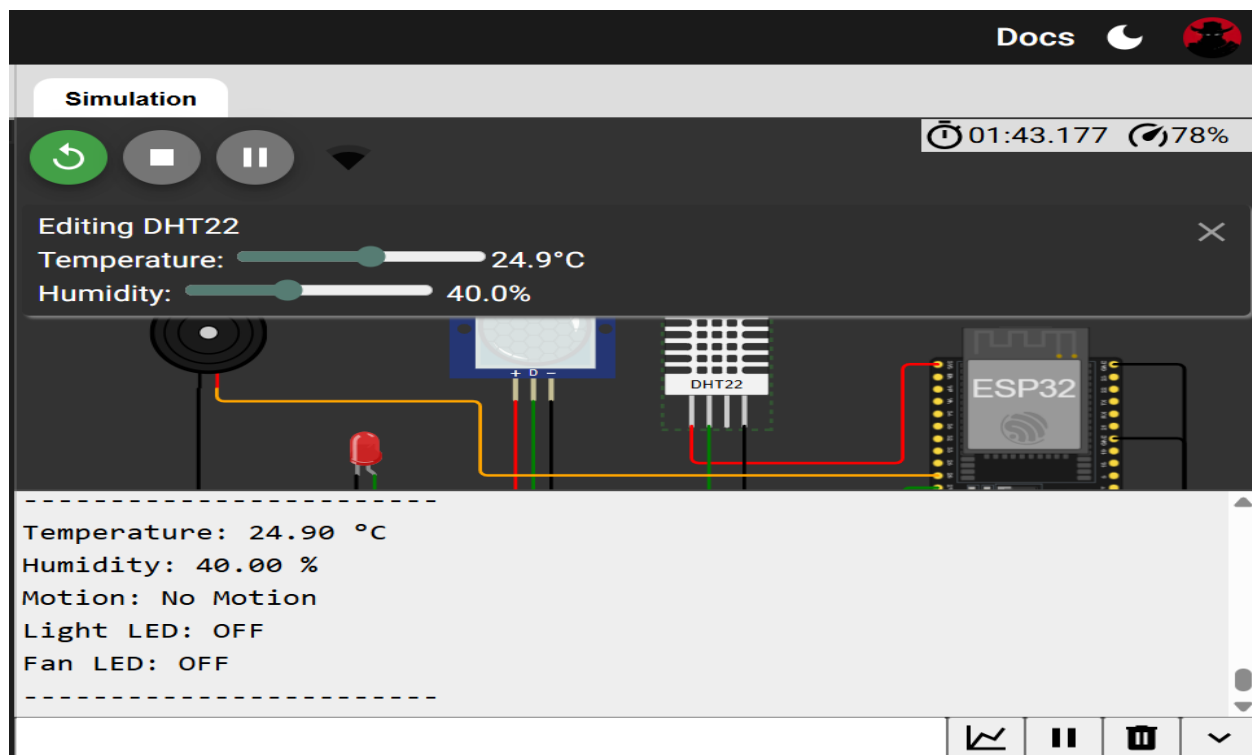
99     if (millis() - lastUpload >= uploadInterval) {
100         ThingSpeak.setField(1, temperature);
101         ThingSpeak.setField(2, humidity);
102         ThingSpeak.setField(3, motion);
103
104         int status = ThingSpeak.writeFields(
105             TS_CHANNEL_ID,
106             TS_WRITE_API_KEY
107         );
108
109         if (status == 200) {
110             Serial.println("Data uploaded to ThingSpeak");
111         } else {
112             Serial.print("ThingSpeak Error: ");
113             Serial.println(status);
114         }
115
116         lastUpload = millis();
117     }
118
119 // ===== Serial Debug =====
120 Serial.println("-----");
121 Serial.print("Temperature: ");
122 Serial.print(temperature);

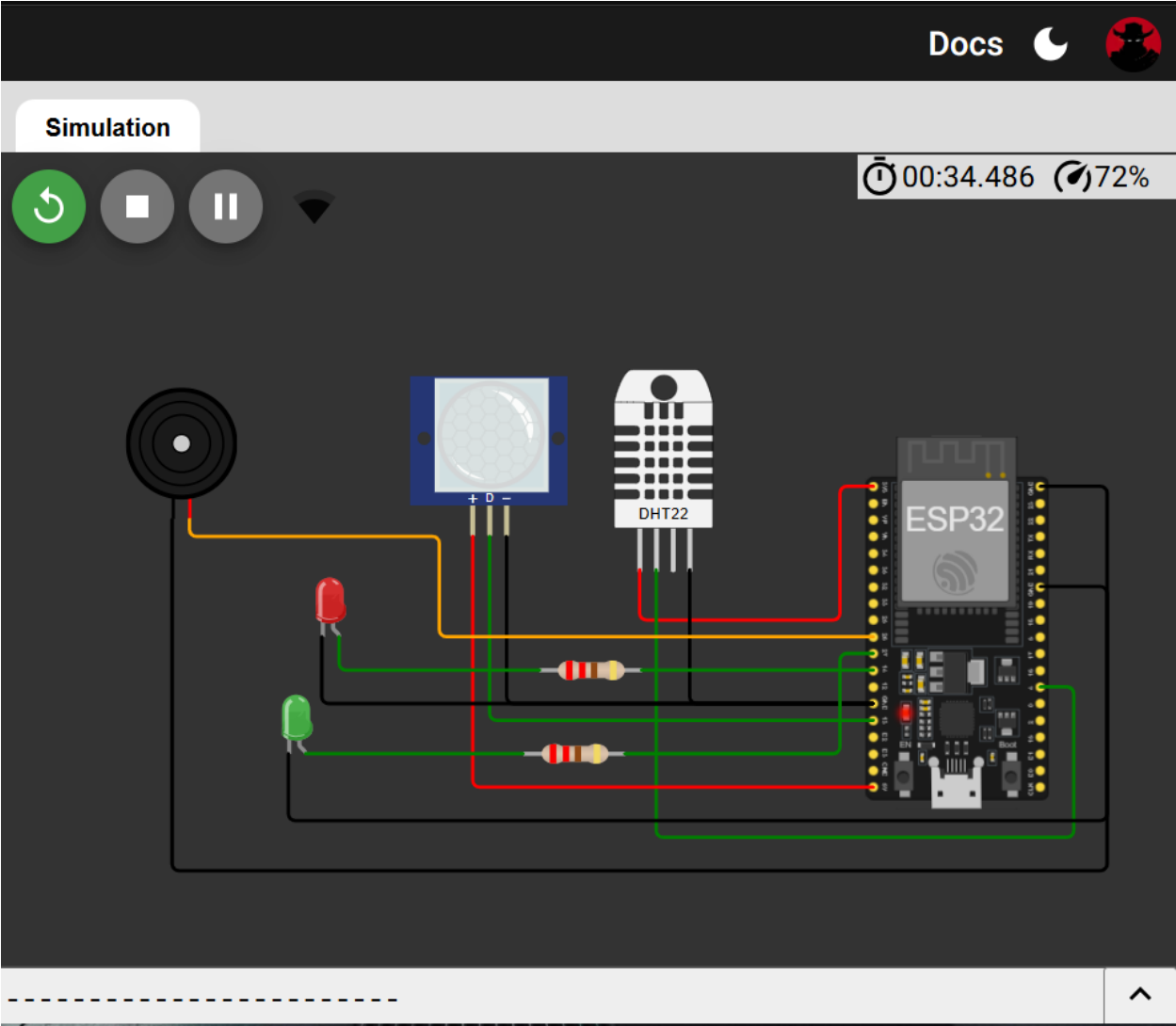
```

```

119 // ===== Serial Debug =====
120 Serial.println("-----");
121 Serial.print("Temperature: ");
122 Serial.print(temperature);
123 Serial.println(" °C");
124
125 Serial.print("Humidity: ");
126 Serial.print(humidity);
127 Serial.println(" %");
128
129 Serial.print("Motion: ");
130 Serial.println(motion ? "Detected" : "No Motion");
131
132 Serial.print("Light LED: ");
133 Serial.println(motion ? "ON" : "OFF");
134
135 Serial.print("Fan LED: ");
136 Serial.println(temperature > 30 ? "ON" : "OFF");
137
138 delay(2000);
139 }

```





Docs

Simulation

02:41.904 99%

Editing DHT22

Temperature: 55.4°C

Humidity: 65.0%

Temperature: 55.40 °C

Humidity: 65.00 %

Motion: No Motion

Light LED: OFF

Fan LED: ON

02:54.898 64%

Editing DHT22

Temperature: 55.4°C

Humidity: 65.0%

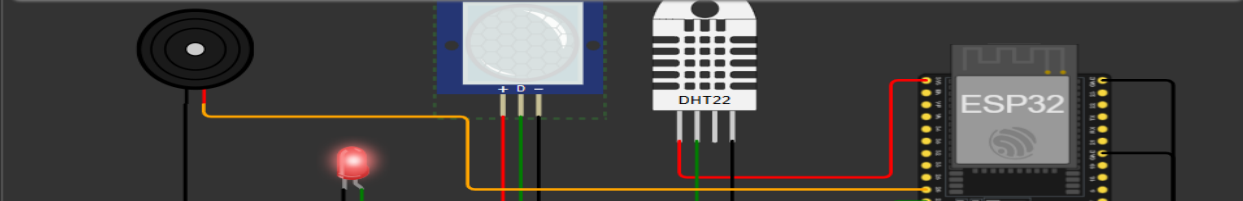
Docs

Simulation

03:18.372 93%

PIR Motion Sensor

Simulate motion



Temperature: 28.50 °C

Humidity: 45.00 %

Motion: Detected

Light LED: ON

Fan LED: OFF

Smart IoT Dashboard

LIVE

Temperature

28.5 °C

Humidity

45.0 %

Motion

Detected

ThingSpeak + Firebase Hybrid IoT System

Temperature: 50.40 °C

Humidity: 70.50 %

Motion: Detected

Light LED: ON

Fan LED: ON

